COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/02/26	**DATE SUBMITTED:**	09/07/26
NAME:	Cruz, Elijah J	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7 SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT

1. PROCEDURE OUTPUT

What procedures did you perform?

CODE (PROVIDED IN THE LAST LECTURE)

Main.cpp

```cpp
//Cruz, Elijah J. - CPE21S2
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);
int main(){

   int unsortedArr[maxSize];

   // use time (0) as seed for random values
   srand(time(0));

   //populate the array with random values
   for(int i = 0; i < maxSize; i++){
       unsortedArr[i] = rand()%100;
   }

   //display the randomly generated array
   std::cout<<"\n unsortedArr: \n";
   displayArr(unsortedArr);

   //std::cout<<"\n bubble sort: \n";
   //bubbleSort(unsortedArr, maxSize);
   //displayArr(unsortedArr);

   //std::cout<<"\n selection sort: \n";
   //selectionSort(unsortedArr, maxSize);
   //displayArr(unsortedArr);

   std::cout<<"\n insertion sort: \n";
   insertionSort(unsortedArr, maxSize);
   displayArr(unsortedArr);

   return 0;
}

void displayArr(int arr[]) {
   for (int i = 0; i < maxSize; i++) {
       std::cout << arr[i] << " ";
   }
   std::cout << std::endl;
}
```

sortingAlgo.h

```
//Cruz, Elijah J. - CPE21S2
#ifndef SORTING_ALGORITHM_SORTINGALGO_H
#define SORTING_ALGORITHM_SORTINGALGO_H

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
   //step1: for i = 0 to N - 1 repeat inner loop
   for (int i = 0; i < arrSize; i++) {
       //step 2 inner loop that for K = i + 1 to N - 1 repeat
       for (int j = i + 1; j < arrSize; j++) {
           //step 3 swap if the arr[j] > arr[i]
           if (arr[i] > arr[j]) {
               //swap arr[j] array[i]
               std::swap(arr[i], arr[j]);
           }
       }
   }
}

template <typename T>
//array, current position, size of array
int  routineSmallest(T arr[], int K, const int arrSize) {
   int position = K;
   T smallest = arr[K];
   for (int j = K + 1; j < arrSize; j++) {
       if (arr[j] < smallest) {
           smallest = arr[j];
           position = j;
       }
   }
   return position;
}

template <typename T>
// POS = POSITION
void selectionSort(T arr[], size_t arrSize) {
   int POS, temp, pass;
   for(pass = 0; pass < arrSize; pass++) {
       POS = routineSmallest(arr, pass, arrSize);
       //swapping
       if (POS != pass) {
           temp = arr[pass];
           arr[pass] = arr[POS];
           arr[POS] = temp;

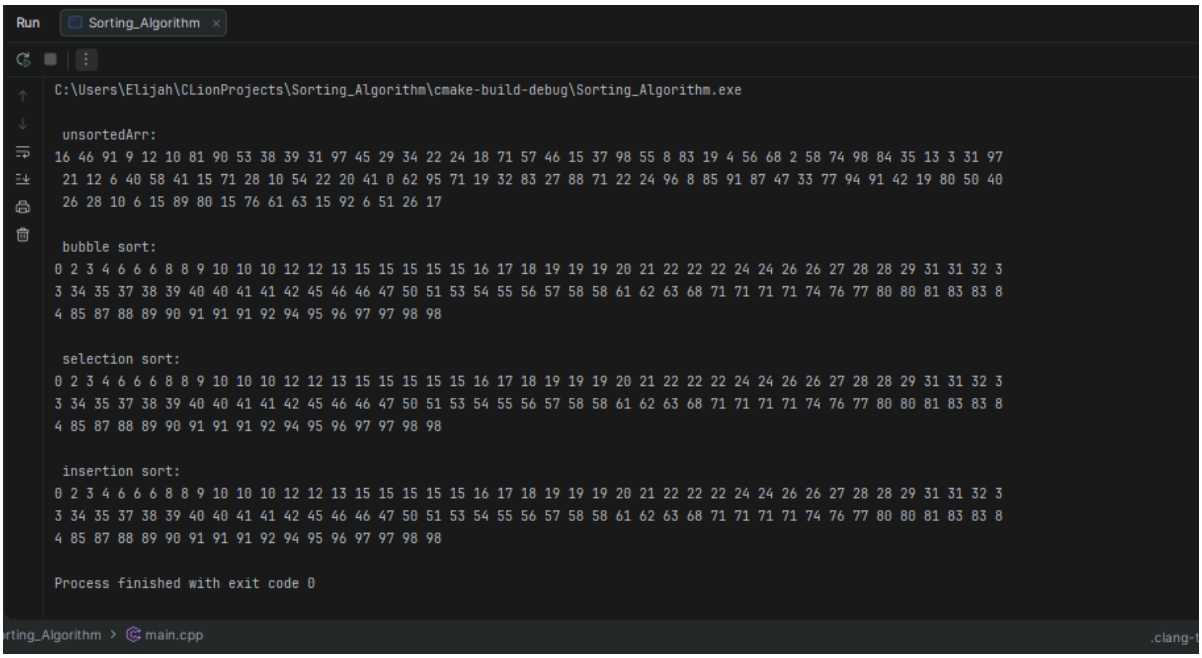
           //for verification and counting of number ofg passes
           pass++;
       }
   }
}

template <typename T>
void insertionSort(T arr[], size_t arrSize) {
   int K = 0, J, temp;

   while (K < arrSize) {
       temp = arr[K];
       J = K - 1;
       while (J >= 0 && temp < arr[J]) {
           arr[J+1] = arr[J];
           J--;
       }

       arr[J+1] = temp;
       K++;
   }
}
#endif //SORTING_ALGORITHM_SORTINGALGO_H
```

OUTPUT



Printed output contains different types of sorting techniques used for the randomly generated array of elements. Of which, all outputs of the different sorting techniques are the same.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

PSEUDOCODE

START of program

CONSTANT maxSize = 100 CONSTANT candidCount = 5

DECLARE votes[maxSize] DECLARE counts[candidCount] = all zeros

// storage for votes and per-candidate tallies

FOR i = 0 TO maxSize - 1 votes[i] = random number between 1 and 5 END FOR

// generate 100 random votes for candidates 1-5

DISPLAY votes (unsorted)

// show the raw votes before sorting

SORT votes in ascending order

// group identical votes together using bubble sort

DISPLAY votes (sorted)

// show the votes now grouped by candidate

FOR c = 1 TO candidCount total = 0 FOR i = 0 TO maxSize - 1 IF votes[i] == c THEN total = total + 1 END IF END FOR counts[c - 1] = total END FOR

// tally how many votes each candidate received

DISPLAY vote counts for each candidate

// show each candidate's total

winnerIndex = 0 FOR c = 1 TO candidCount - 1 IF counts[c] > counts[winnerIndex] THEN winnerIndex = c END IF END FOR

// find the candidate with the highest total

DISPLAY winnerIndex + 1 as the winning candidate DISPLAY counts[winnerIndex] as the winning vote count // announce the winner and their vote count

END of program

CODE

Main.cpp

```
//Cruz, Elijah J. - CPE21S2
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
const int candidCount = 5;

void displayArr(int arr[]);
void countVotes(int arr[], int counts[]);
int findWinner(int counts[]);

int main(){
```

```
   int votes[maxSize];
   int counts[candidCount] = {0};
   srand(time(0));

   for(int i = 0; i < maxSize; i++){
       votes[i] = rand() % 5 + 1;
   }

   //display the randomly generated votes
   std::cout<<"\n unsorted votes: \n";
   displayArr(votes);

   bubbleSort(votes, maxSize);
   std::cout<<"\n sorted votes: \n";
   displayArr(votes);

   //WE search the sorted array and tally the votes per candidate
   countVotes(votes, counts);
   for(int c = 0; c < candidCount; c++){
       std::cout<<"candidate "<<(c+1)<<": "<<counts[c]<<" votes\n";
   }

   //find the candidate with the highest number of votes
   int winner = findWinner(counts);
   std::cout<<"The winning candidate is candidate "<< winner + 1 <<" with "<< counts[winner] <<"
votes\n";

   return 0;
}

void displayArr(int arr[]) {
   for (int i = 0; i < maxSize; i++) {
       std::cout << arr[i] << " ";
   }
   std::cout << std::endl;
}

void countVotes(int arr[], int counts[]) {
   for (int c = 1; c <= candidCount; c++) {
       int total = 0;
           for (int i = 0; i < maxSize; i++) {
               if (arr[i] == c) {
               total++;
           }
       }
       counts[c-1] = total;
   }
}

int findWinner(int counts[]) {
   int winnerIndex = 0;
       for (int c = 1; c < candidCount; c++) {
           if (counts[c] > counts[winnerIndex]) {
           winnerIndex = c;
       }
   }
   return winnerIndex;
}
```

Contains the code for main.cpp file responsible for the main functions such as displaying the sorted, unsorted arrays and the loops for the processes of counting the number of votes to choose the winner with the highest votes for this supplementary activity.

sortingAlgo.h

```
//Cruz, Elijah J. - CPE21S2
#ifndef SORTING_ALGORITHM_SORTINGALGO_H
#define SORTING_ALGORITHM_SORTINGALGO_H
```

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
   //step1: for i = 0 to N - 1 repeat inner loop
   for (int i = 0; i < arrSize; i++) {
       //step 2 inner loop that for K = i + 1 to N - 1 repeat
       for (int j = i + 1; j < arrSize; j++) {
           //step 3 swap if the arr[j] > arr[i]
           if (arr[i] > arr[j]) {
               //swap arr[j] array[i]
               std::swap(arr[i], arr[j]);
           }
       }
   }
}

template <typename T>
//array, current position, size of array
int  routineSmallest(T arr[], int K, const int arrSize) {
   int position = K;
   T smallest = arr[K];
   for (int j = K + 1; j < arrSize; j++) {
       if (arr[j] < smallest) {
           smallest = arr[j];
           position = j;
       }
   }
   return position;
}

template <typename T>
// POS = POSITION
void selectionSort(T arr[], size_t arrSize) {
   int POS, temp, pass;
   for(pass = 0; pass < arrSize; pass++) {
       POS = routineSmallest(arr, pass, arrSize);
       //swapping
       if (POS != pass) {
           temp = arr[pass];
           arr[pass] = arr[POS];
           arr[POS] = temp;

           //for verification and counting of number ofg passes
           pass++;
       }
   }
}

template <typename T>
void insertionSort(T arr[], size_t arrSize) {
   int K = 0, J, temp;

   while (K < arrSize) {
       temp = arr[K];
       J = K - 1;
       while (J >= 0 && temp < arr[J]) {
           arr[J+1] = arr[J];
           J--;
       }

       arr[J+1] = temp;
       K++;
   }
}
#endif //SORTING_ALGORITHM_SORTINGALGO_H
```

Contains the code and main function for the bubble sort function that groups all the matching votes together before it is counted. It repeatedly steps through the array and compares the elements and swaps them if they are out of order.

Output Console Showing Sorted Array

```
sorted votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

```
sorted votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

```
sorted votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count

candidate 1: 24 votes
candidate 2: 8 votes
candidate 3: 31 votes
candidate 4: 18 votes
candidate 5: 19 votes

candidate 1: 17 votes
candidate 2: 17 votes
candidate 3: 20 votes
candidate 4: 20 votes
candidate 5: 26 votes

candidate 1: 22 votes
candidate 2: 21 votes
candidate 3: 27 votes
candidate 4: 14 votes
candidate 5: 16 votes

Count Result of Algorithm

candidate 1: 24 votes
candidate 2: 8 votes
candidate 3: 31 votes
candidate 4: 18 votes
candidate 5: 19 votes

The winning candidate is candidate 3 with 31 votes

candidate 1: 17 votes
candidate 2: 17 votes

candidate 3: 20 votes
candidate 4: 20 votes
candidate 5: 26 votes

The winning candidate is candidate 5 with 26 votes

candidate 1: 22 votes
candidate 2: 21 votes
candidate 3: 27 votes
candidate 4: 14 votes
candidate 5: 16 votes

The winning candidate is candidate 3 with 27 votes

Question: Was your developed vote counting algorithm effective? Why or why not?
Yes, as the developed vote counting algorithm effectively sorts the vote counts that allows the program to seamlessly count the votes and create a conclusion at a very fast pace.

Conclusion:
The activity focuses on learning how to integrate and apply sorting techniques involving large quantities of data in order to process this said datas more efficiently allowing us to save more resources and time. But, as for this supplementary activity, we cannot conclude or make assertions that one of the said techniques is the fastest as the activity involves only a small randomly generated set of array elements which were sorted and used to create conclusions of which candidates had the highest amount of votes. As for this activity, I have more areas for improvement and that practice is a necessary step for me to be able to fully grasp how to apply sorting techniques for the following activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI provided a guide and grammar correction**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.